

Base Teórica: Fase III

Auditoría de Aplicaciones Web y Generación de Reportes

Curso de Seguridad Informática

21 de junio de 2026

Resumen

Este documento proporciona la base teórica fundamental para la **Fase III** del plan de trabajo de la Suite de Auditoría de Seguridad. Se profundiza en los conceptos clave de la capa de aplicación (HTTP), técnicas de rastreo ofensivo (Crawling y Fuzzing), inyección de código (SQL Injection), vulnerabilidades de ejecución del lado del cliente y del lado del servidor (XSS, LFI/RFI) y la metodología de reporte técnico y ejecutivo.

Índice

1. Introducción a la Capa de Aplicación y Protocolo HTTP	2
1.1. Estructura de una Petición HTTP	2
1.2. Estructura de una Respuesta HTTP	2
2. Grupo 1: Rastreo y Mapeo Web (Web Crawling & Fuzzing)	2
2.1. Web Crawling (Rastreo Web)	3
2.2. Web Fuzzing (Descubrimiento de Directorios)	3
3. Grupo 3: Inyección de Código SQL (SQLi)	3
3.1. Lógica del Ataque	3
3.2. Tipos de SQLi	4
4. Grupo 4: Vulnerabilidades del Lado del Cliente y del Servidor (XSS y LFI)	4
4.1. Cross-Site Scripting (XSS)	4
4.2. Inclusión de Archivos Locales (LFI) y Directory Traversal	4
5. Grupo 2: Orquestación, Consolidación y Generación de Reportes	4
5.1. El Contrato de Datos (schema_resultados.json)	5
5.2. Formatos de Reporte	5
6. Diagrama de Flujo del Proceso de Auditoría de la Fase III	5
7. Consideraciones Éticas y Legales	6

1. Introducción a la Capa de Aplicación y Protocolo HTTP

En las fases anteriores, la auditoría se concentró en las capas de red y de transporte (descubrimiento de hosts y escaneo de puertos), así como en la enumeración de servicios y protocolos de infraestructura (SMB, SSH, FTP). La Fase III desplaza el foco hacia la capa de aplicación, específicamente al protocolo HTTP (Hypertext Transfer Protocol).

HTTP es un protocolo sin estado (stateless) basado en un modelo de petición-respuesta. Para comprender y explotar vulnerabilidades web, los auditores deben analizar en detalle la estructura de estos mensajes.

1.1. Estructura de una Petición HTTP

Una petición HTTP estándar consta de tres partes principales:

1. **Línea de petición:** Contiene el método HTTP (GET, POST, PUT, DELETE), la URI del recurso solicitado y la versión del protocolo (ej., HTTP/1.1).
2. **Cabeceras (Headers):** Metadatos que describen al cliente, el host solicitado, las cookies de sesión (Cookie), el tipo de contenido aceptado, entre otros.
3. **Cuerpo del mensaje (Body):** Opcional. Contiene los datos enviados al servidor (generalmente en métodos POST o PUT, como datos de formularios o JSON).

1.2. Estructura de una Respuesta HTTP

La respuesta del servidor incluye:

1. **Línea de estado:** Versión del protocolo y código de estado HTTP (ej., 200 OK, 404 Not Found, 500 Internal Server Error).
2. **Cabeceras:** Detalles del servidor web, tipo de contenido (Content-Type), directivas de seguridad (Content-Security-Policy), etc.
3. **Cuerpo del mensaje:** El código HTML, archivos CSS, JS, imágenes o datos en formato JSON/XML.

2. Grupo 1: Rastreo y Mapeo Web (Web Crawling & Fuzzing)

El objetivo del primer subgrupo en la Fase III es realizar el reconocimiento activo de la aplicación web, mapeando su superficie de ataque mediante dos técnicas primordiales:

2.1. Web Crawling (Rastreo Web)

Un *crawler* (o araña) es un script automatizado que simula el comportamiento de un usuario navegando por el sitio web. El flujo lógico consiste en:

1. Realizar una petición inicial a la URL raíz del objetivo.
2. Analizar el código HTML de la respuesta (usando librerías como `BeautifulSoup`) buscando etiquetas de hipervínculos como `<a href="...">`.
3. Normalizar los enlaces relativos a rutas absolutas.
4. Visitar de manera recursiva las nuevas URLs descubiertas pertenecientes al mismo dominio, manteniendo un registro de páginas visitadas para evitar bucles infinitos.
5. Extraer datos de interés expuestos en el código, tales como comentarios de los desarrolladores (que a menudo contienen credenciales u pistas de diseño), direcciones de correo electrónico, y scripts de JavaScript de terceros.

2.2. Web Fuzzing (Descubrimiento de Directorios)

No todos los recursos de un servidor web son enlazados públicamente en su HTML. Muchos directorios administrativos o archivos de respaldo se encuentran “ocultos”. El *fuzzing* web consiste en forzar peticiones HTTP hacia el servidor utilizando una lista predefinida de palabras comunes (diccionario), evaluando el código de respuesta devuelto:

- Código **200 OK** o **403 Forbidden**: Indican la existencia del recurso.
- Código **404 Not Found**: Indica que el recurso no existe.

3. Grupo 3: Inyección de Código SQL (SQLi)

La Inyección SQL (SQLi) es una de las vulnerabilidades más críticas en aplicaciones web. Ocurre cuando datos provistos por el usuario son concatenados directamente en una consulta a la base de datos sin una sanitización o parametrización previa.

3.1. Lógica del Ataque

Considere un script de backend vulnerable que realiza la siguiente consulta:

```
1 SELECT * FROM usuarios WHERE username = 'USER_INPUT' AND password = 'PASSWORD_INPUT';
```

Si un atacante introduce en `USER_INPUT` el payload `' OR '1'='1'`, la consulta se transforma en:

```
1 SELECT * FROM usuarios WHERE username = '' OR '1'='1' AND password = 'PASSWORD_INPUT';
```

Dado que la condición `'1'='1'` siempre es verdadera, el motor de base de datos devolverá los registros de la tabla, saltándose el control de autenticación.

3.2. Tipos de SQLi

- **In-Band (Basada en errores o Union):** El atacante puede ver los resultados de la inyección o los mensajes de error directamente en la respuesta HTML.
- **Inferencial (Ciega / Blind):** La aplicación no devuelve datos de la BD ni errores explícitos. El atacante debe inferir la información haciendo preguntas de verdadero/falso al servidor y midiendo el comportamiento de la aplicación (ej., basadas en respuestas booleanas o retrasos de tiempo inducidos).

4. Grupo 4: Vulnerabilidades del Lado del Cliente y del Servidor (XSS y LFI)

Este subgrupo audita dos clases importantes de vulnerabilidades web:

4.1. Cross-Site Scripting (XSS)

XSS ocurre cuando una aplicación web recibe datos de entrada no confiables y los incluye en la página web resultante sin validación o escape adecuados, lo que permite la ejecución de código JavaScript en el navegador del usuario víctima.

- **XSS Reflejado (Non-Persistent):** El script malicioso se incluye en la petición HTTP (por ejemplo, en un parámetro URL como `?search=<script>...`) y el servidor lo “refleja” inmediatamente en la respuesta.
- **XSS Almacenado (Persistent):** El payload se almacena de forma permanente en la base de datos del servidor (como un comentario en un foro). Cada vez que cualquier usuario visite esa sección, el script se ejecutará.

4.2. Inclusión de Archivos Locales (LFI) y Directory Traversal

LFI ocurre cuando una aplicación web permite a un usuario controlar qué archivo se cargará o procesará en el servidor. Si el parámetro no está restringido a una lista blanca, el atacante puede emplear la técnica de *Directory Traversal* (uso de caracteres `../`) para leer archivos internos sensibles del sistema operativo.

Un ejemplo típico de payload LFI para sistemas Unix:

```
1 http://objetivo.local/ver_archivo.php?page=../../../../etc/passwd
```

5. Grupo 2: Orquestación, Consolidación y Generación de Reportes

La última etapa del ciclo de vida de una prueba de penetración o auditoría es la comunicación de hallazgos. El Grupo 2 es responsable de centralizar la ejecución de todos los módulos y formatear los resultados unificados bajo un contrato de datos común.

5.1. El Contrato de Datos (schema_resultados.json)

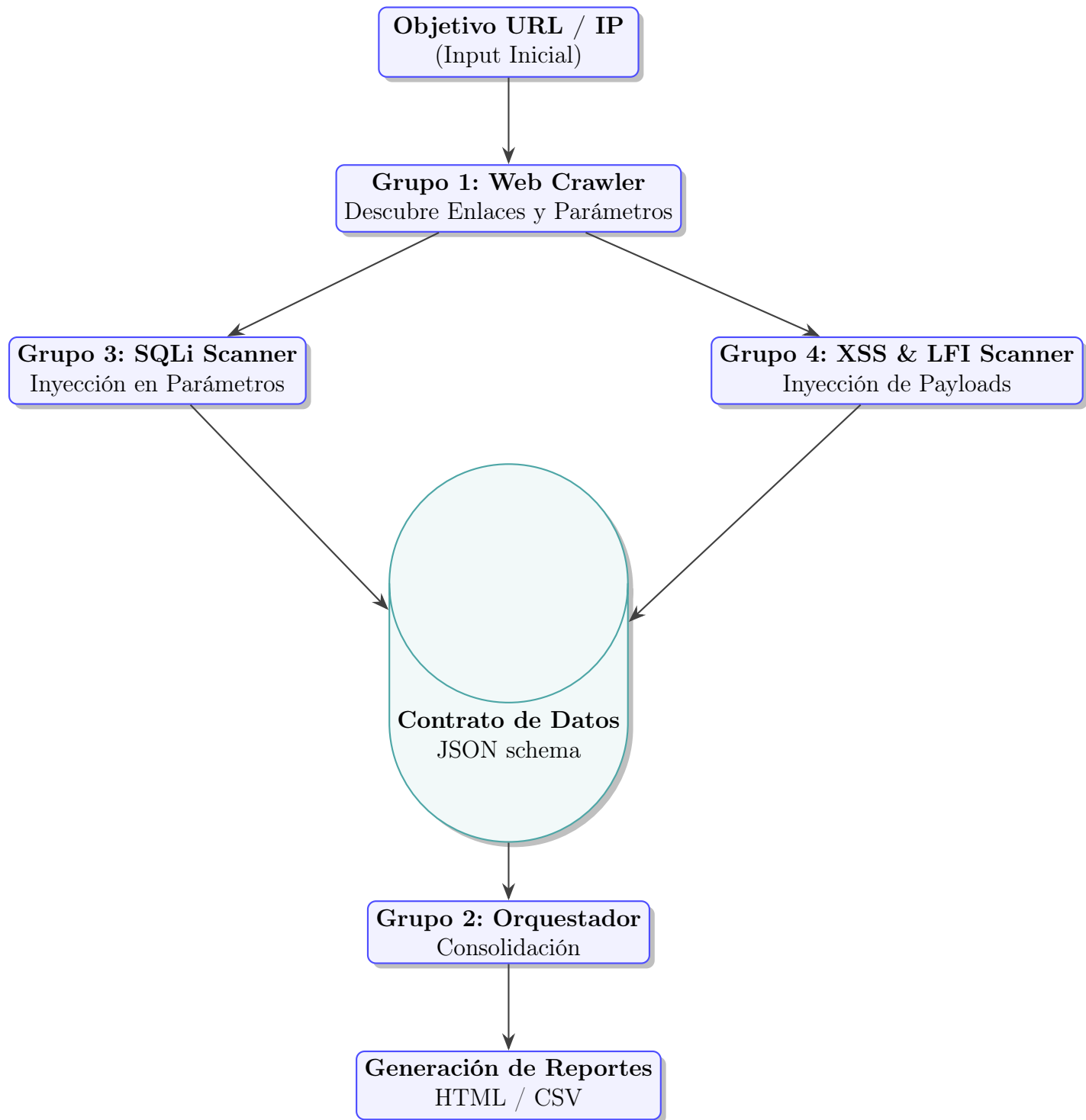
Para lograr que la suite sea verdaderamente modular y escalable, las salidas de cada fase de auditoría deben respetar estrictamente un esquema JSON. Esto permite que el motor de reportes consuma los datos de forma consistente sin importar qué herramienta o grupo los generó.

5.2. Formatos de Reporte

- **CSV (Tabular):** Ideal para el análisis posterior por parte del equipo técnico de administración de sistemas o importación en sistemas SIEM/hojas de cálculo.
- **HTML (Ejecutivo / Visual):** Ofrece una interfaz de usuario limpia, con gráficos interactivos y tablas ordenadas que permiten a los tomadores de decisiones comprender el estado de seguridad de sus activos de un vistazo.

6. Diagrama de Flujo del Proceso de Auditoría de la Fase III

A continuación se ilustra cómo interactúan los diferentes módulos de la Fase III durante una ejecución integrada en el orquestador principal:



7. Consideraciones Éticas y Legales

“Un gran poder conlleva una gran responsabilidad.”

Las herramientas desarrolladas en esta fase simulan ataques cibernéticos reales de nivel

de aplicación. Los estudiantes deben recordar que:

1. Está prohibido ejecutar estas herramientas contra sistemas de terceros sin consentimiento explícito y formal (autorización de pen-testing por escrito).
2. Los escáneres automáticos de vulnerabilidades web consumen recursos de red y procesamiento significativos en el backend, pudiendo provocar una Denegación de Servicio (DoS) involuntaria.
3. Para fines educativos, todas las pruebas de esta fase deben realizarse sobre laboratorios locales controlados, tales como la máquina virtual *Metasploitable 2* o aplicaciones web deliberadamente vulnerables como *DVWA (Damn Vulnerable Web App)*.